

SSN College of Engineering

LABORATORY RECORD

Subject: Web Application Development Laboratory

Exercise 6: College Symposium Management System using Vue.js

SUBMITTED BY

Name: Vijayaraaghavan K S

Register Number: 3122247001066

Branch: Department of CSE

Semester / Year: V Semester / III Year

Subject Code: ICS1511 Batch: 2024–2029

1. Problem Description

Scenario

A college is organizing a three-day technical symposium (TechNova) with coding contests, workshops, paper presentations, quizzes, and cultural programs. The application lets students browse, filter, search, and register for events, with seat counts and registration status updating live.

Technology Stack

- Vue 3 with `<script setup>` (Vite scaffold)
- Reusable components: `App.vue`, `Header.vue`, `EventList.vue`, `EventCard.vue`, `EventDetails.vue`, `RegistrationForm.vue`, `Footer.vue`
- Props for parent→child data flow, computed for filtering/searching
- `:class` and `:style` for category and seat-based dynamic styling

Requirement Checklist (all 18 satisfied)

1. Seven components created as specified.
2. Event data (9 fields per event) stored in `App.vue`.
3. Data passed down to children via props.
4. Events rendered with `v-for` in `EventList.vue`.
5. `:class` binding colors each `EventCard` by category.
6. `v-if` / `v-else-if` / `v-else` renders Open / Almost Full / Closed.
7. Available seats computed dynamically (`max - registered`).
8. “View Details” button opens `EventDetails.vue` for that event.
9. `EventDetails.vue` shows a Register button only when open and seats remain.
10. Clicking Register reveals `RegistrationForm.vue`.
11. Submission validates all fields, increments registered count, decrements seats, shows a success message, and status auto-updates on full.
12. `:style` sets the title font size by seat count (`>20` large, `10–20` medium, `<10` small).
13. Category filter (All / Technical / Cultural / Workshop / Competition).
14. Search box filters events by name.
15. computed properties implement both filtering and searching together.
16. Event handling covers search, category selection, view details, registration, and form submission.
17. Four styling approaches used: normal CSS (`Header.vue`), `<style scoped>` (`EventCard.vue`), `:class` (category color), `:style` (dynamic font size).

18. Adding a new event only requires adding an object to the events array in App.vue — no component code changes needed.

2. Source Code

File: App.vue

```
<script setup>
import { computed, reactive, ref } from 'vue'
import Header from './components/Header.vue'
import EventList from './components/EventList.vue'
import EventDetails from './components/EventDetails.vue'
import Footer from './components/Footer.vue'

// Event data lives here in App.vue and is passed down to children via props.
const events = reactive([
  {
    id: 'EV01',
    name: 'Code Sprint',
    category: 'Technical',
    date: '2026-09-10',
    venue: 'CSE Lab 1',
    registrationFee: 100,
    maxParticipants: 60,
    registeredParticipants: 15,
    status: 'Registration Open'
  }
  // ... EV02-EV08 follow the same shape (Technical, Cultural, Workshop,
  // Competition)
])

const categories = ['All Events', 'Technical', 'Cultural', 'Workshop',
  'Competition']
const activeCategory = ref('All Events')
const searchTerm = ref('')
const selectedEventId = ref(null)

// Computed property: filters by category and searches by event name.
const filteredEvents = computed(() => {
  return events.filter((event) => {
    const matchesCategory =
      activeCategory.value === 'All Events' || event.category ===
      activeCategory.value
    const matchesSearch = event.name
      .toLowerCase()
      .includes(searchTerm.value.trim().toLowerCase())
    return matchesCategory && matchesSearch
  })
})
})
```

```

const selectedEvent = computed(
  () => events.find((event) => event.id === selectedEventId.value) || null
)

function handleSelectCategory(category) {
  activeCategory.value = category
}

function handleSearch(term) {
  searchTerm.value = term
}

function handleViewDetails(eventId) {
  selectedEventId.value = eventId
}

function handleCloseDetails() {
  selectedEventId.value = null
}

function handleRegister({ eventId }) {
  const event = events.find((e) => e.id === eventId)
  if (!event) return

  event.registeredParticipants += 1

  const availableSeats = event.maxParticipants - event.registeredParticipants
  if (availableSeats <= 0) {
    event.status = 'Registration Closed'
  } else if (availableSeats <= 10) {
    event.status = 'Almost Full'
  } else {
    event.status = 'Registration Open'
  }
}
</script>

<template>
  <div class="app-shell">
    <Header
      :categories="categories"
      :active-category="activeCategory"
      :search-term="searchTerm"
      @select-category="handleSelectCategory"
      @search="handleSearch"
    />

    <EventDetails
      v-if="selectedEvent"
      :event="selectedEvent"
      @register="handleRegister"
      @close="handleCloseDetails"
    />
  </div>
</template>

```

```

    <EventList :events="filteredEvents" @view-details="handleViewDetails" />

    <Footer />
  </div>
</template>

<style scoped>
.app-shell {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}
</style>

```

File: Header.vue (normal, non-scoped CSS)

```

<script setup>
defineProps({
  categories: { type: Array, required: true },
  activeCategory: { type: String, required: true },
  searchTerm: { type: String, required: true }
})

defineEmits(['select-category', 'search'])
</script>

<template>
  <header class="site-header">
    <div class="brand">
      <h1>TechNova Symposium</h1>
      <p>College Symposium Management System</p>
    </div>

    <div class="controls">
      <input
        class="search-box"
        type="text"
        placeholder="Search events by name..."
        :value="searchTerm"
        @input="$emit('search', $event.target.value)"
      />

      <select
        class="category-filter"
        :value="activeCategory"
        @change="$emit('select-category', $event.target.value)"
      >
        <option v-for="cat in categories" :key="cat" :value="cat">{{ cat }}</option>
      </select>
    </div>
  </header>

```

```

</template>

<!-- Normal (non-scoped) CSS, applies globally as required for Header.vue -->
<style>
.site-header {
  background: #23264a;
  color: #fff;
  padding: 20px 32px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
</style>

```

File: EventList.vue

```

<script setup>
import EventCard from './EventCard.vue'

defineProps({ events: { type: Array, required: true } })
defineEmits(['view-details'])
</script>

<template>
<section class="event-list">
  <p v-if="events.length === 0" class="no-events">
    No events match your search / filter.
  </p>

  <EventCard
    v-for="event in events"
    :key="event.id"
    :event="event"
    @view-details="$emit('view-details', $event)"
  />
</section>
</template>

<style scoped>
.event-list {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 20px;
  padding: 24px 32px;
}
</style>

```

File: EventCard.vue (<style scoped> + :class + :style)

```
<script setup>
import { computed } from 'vue'

const props = defineProps({ event: { type: Object, required: true } })
defineEmits(['view-details'])

const availableSeats = computed(
  () => props.event.maxParticipants - props.event.registeredParticipants
)

const status = computed(() => {
  if (availableSeats.value <= 0) return 'Registration Closed'
  if (availableSeats.value <= 10) return 'Almost Full'
  return 'Registration Open'
})

// Dynamic font size (:style) based on available seats.
const titleStyle = computed(() => {
  const seats = availableSeats.value
  if (seats > 20) return { fontSize: '1.6rem' }
  if (seats >= 10) return { fontSize: '1.2rem' }
  return { fontSize: '0.95rem' }
})
</script>

<template>
  <div class="event-card" :class="event.category.toLowerCase()">
    <h3 class="event-title" :style="titleStyle">{{ event.name }}</h3>

    <span
class="badge" :class="event.category.toLowerCase()">{{ event.category }}</
span>

    <p class="meta"><strong>Seats:</strong> {{ availableSeats > 0 ?
availableSeats : 0 }} / {{ event.maxParticipants }} available</p>

    <p v-if="status === 'Registration Open'" class="status open">Registration
Open</p>
    <p v-else-if="status === 'Almost Full'" class="status almost">Almost
Full</p>
    <p v-else class="status closed">Registration Closed</p>

    <button class="details-btn" @click="$emit('view-details', event.id)">View
Details</button>
  </div>
</template>

<style scoped>
.event-card { background: #fff; border-radius: 10px; padding: 18px; border-
top: 5px solid #999; }
.event-card.technical { border-top-color: #2563eb; }
```

```

.event-card.cultural { border-top-color: #db2777; }
.event-card.workshop { border-top-color: #059669; }
.event-card.competition { border-top-color: #d97706; }
.status.open { color: #059669; }
.status.almost { color: #d97706; }
.status.closed { color: #dc2626; }
</style>

```

File: EventDetails.vue

```

<script setup>
import { computed, ref } from 'vue'
import RegistrationForm from './RegistrationForm.vue'

const props = defineProps({ event: { type: Object, required: true } })
const emit = defineEmits(['register', 'close'])

const showForm = ref(false)
const successMessage = ref('')

const availableSeats = computed(
  () => props.event.maxParticipants - props.event.registeredParticipants
)

const status = computed(() => {
  if (availableSeats.value <= 0) return 'Registration Closed'
  if (availableSeats.value <= 10) return 'Almost Full'
  return 'Registration Open'
})

const canRegister = computed(
  () => status.value !== 'Registration Closed' && availableSeats.value > 0
)

function openForm() {
  showForm.value = true
  successMessage.value = ''
}

function handleFormSubmit(studentData) {
  emit('register', { eventId: props.event.id, studentData })
  showForm.value = false
  successMessage.value = `Thank you, ${studentData.studentName}! You have
  successfully registered for "${props.event.name}".`
}
</script>

<template>
<div class="event-details">
  <button class="close-btn" @click="$emit('close')">&times; Close</button>
  <h2>{{ event.name }}</h2>

```

```

    <p v-if="successMessage" class="success">{{ successMessage }}</p>

    <button v-if="canRegister && !showForm" class="register-btn"
    @click="openForm">Register</button>

    <RegistrationForm v-if="showForm" @submit="handleFormSubmit"
    @cancel="showForm = false" />
  </div>
</template>

```

File: RegistrationForm.vue

```

<script setup>
import { ref } from 'vue'

const emit = defineEmits(['submit', 'cancel'])

const studentName = ref('')
const registerNumber = ref('')
const email = ref('')
const department = ref('')
const errorMessage = ref('')

function handleSubmit() {
  if (!studentName.value.trim() || !registerNumber.value.trim() ||
    !email.value.trim() || !department.value.trim()) {
    errorMessage.value = 'Please fill in all fields before submitting.'
    return
  }

  errorMessage.value = ''
  emit('submit', {
    studentName: studentName.value,
    registerNumber: registerNumber.value,
    email: email.value,
    department: department.value
  })
}
</script>

<template>
  <form class="registration-form" @submit.prevent="handleSubmit">
    <p v-if="errorMessage" class="error">{{ errorMessage }}</p>
    <label>Student Name <input v-model="studentName" type="text" /></label>
    <label>Register Number <input v-model="registerNumber" type="text" /></
  label>
    <label>Email <input v-model="email" type="email" /></label>
    <label>Department <input v-model="department" type="text" /></label>
    <button type="submit">Submit Registration</button>
    <button type="button" @click="$emit('cancel')">Cancel</button>
  </form>
</template>

```

```
</form>
</template>
```

File: Footer.vue

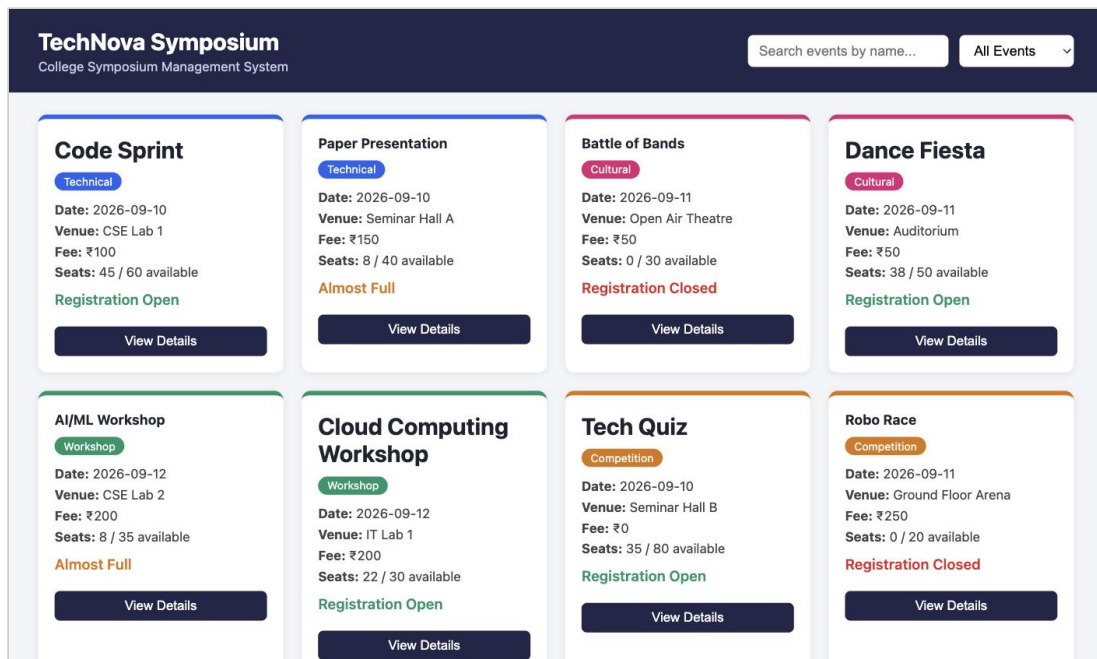
```
<script setup>
const year = new Date().getFullYear()
</script>

<template>
  <footer class="site-footer">
    <p>&copy; {{ year }} TechNova Symposium – College Symposium Management
System</p>
  </footer>
</template>

<style scoped>
.site-footer {
  margin-top: auto;
  background: #23264a;
  color: #c9cbe8;
  text-align: center;
  padding: 16px;
}
</style>
```

3. Output Screenshots

a. Home page – all events, category colors and status



All events

All 8 seeded events render via `v-for`, colored by category through `:class`, with live seat counts and Open/Almost Full/Closed status from `v-if/v-else`.

b. Registration flow – success message and live seat update

Almost Full

Thank you, Vijayaraaghavan K S! You have successfully registered for "AI/ML Workshop".

Register

Code Sprint
Technical
Date: 2026-09-10
Venue: CSE Lab 1
Fee: ₹100
Seats: 45 / 60 available
Registration Open
View Details

Paper Presentation
Technical
Date: 2026-09-10
Venue: Seminar Hall A
Fee: ₹150
Seats: 8 / 40 available
Almost Full
View Details

Battle of Bands
Cultural
Date: 2026-09-11
Venue: Open Air Theatre
Fee: ₹50
Seats: 0 / 30 available
Registration Closed
View Details

Dance Fiesta
Cultural
Date: 2026-09-11
Venue: Auditorium
Fee: ₹50
Seats: 38 / 50 available
Registration Open
View Details

AI/ML Workshop
Workshop
Date: 2026-09-12
Venue: CSE Lab 2
Fee: ₹200
Seats: 7 / 35 available
Almost Full

Cloud Computing Workshop
Workshop
Date: 2026-09-12
Venue: IT Lab 1
Fee: ₹200
Seats: 22 / 30 available

Tech Quiz
Competition
Date: 2026-09-10
Venue: Seminar Hall B
Fee: ₹0
Seats: 35 / 80 available
Registration Open

Robo Race
Competition
Date: 2026-09-11
Venue: Ground Floor Arena
Fee: ₹250
Seats: 0 / 20 available
Registration Closed

Registration success

Filling and submitting the registration form for “AI/ML Workshop” increased registeredParticipants from 27 to 28, dropped available seats from 8 to 7, and displayed the success message — all without a page reload.

c. Search – filtering by event name

TechNova Symposium
College Symposium Management System

quiz

All Events

Tech Quiz
Competition
Date: 2026-09-10
Venue: Seminar Hall B
Fee: ₹0
Seats: 35 / 80 available
Registration Open
View Details

© 2026 TechNova Symposium - College Symposium Management System

Search filter

Typing “quiz” in the search box filters the grid down to “Tech Quiz” via the `filteredEvents` computed property, which combines the active category and the search term in one pass.

4. Learning Outcomes

This exercise demonstrated how a single source of truth (events in `App.vue`) can drive an entire UI purely through props and events, with no direct DOM manipulation. computed properties proved essential for keeping filtering, searching, and seat/status calculations automatically in sync with the underlying reactive data. Mixing four different styling techniques in one app (global CSS, scoped CSS, `:class`, `:style`) clarified when each is appropriate: global CSS for shared chrome like the header, scoped CSS for component-local styling, `:class` for discrete category-based styling, and `:style` for a continuously varying value like font size. Finally, the requirement that “adding a new event only touches data” validated that the component design is genuinely reusable.